

Aula 6 - Consumo de APIs REST com HttpClient

Duração: 4 horas (1h teórica + 3h prática)

Parte Teórica (1 hora)

1. Revisão da Aula Anterior (15 min)

- Discussão sobre:
 - Configuração de rotas e navegação programática.
 - Dúvidas sobre parâmetros de rota (`ActivatedRoute`).

2. Introdução ao HttpClient (25 min)

Conceitos Fundamentais

- **HttpClientModule:** Módulo para requisições HTTP.
- **Métodos Principais:**

typescript

```
get(url)           // Consulta
post(url, body)    // Criação
put(url, body)     // Atualização
delete(url)        // Exclusão
```

- **Observables:** Fluxos assíncronos (manipulação com `subscribe`).

Interceptores

- **Uso típico:**
 - Adicionar headers (ex: Authorization).
 - Tratar erros globalmente.

3. Boas Práticas (20 min)

- **Serviços dedicados:** Isolar chamadas HTTP em serviços.
- **Tipagem de respostas:** Usar interfaces/types.
- **Tratamento de erros:** `catchError` e feedback ao usuário.

Parte Prática (3 horas)

Atividade: Integração com API JSONPlaceholder

Passo 1: Configuração Inicial (30 min)

1. Importar HttpClientModule no AppModule :

```
typescript

import { HttpClientModule } from '@angular/common/http';

@NgModule({
  imports: [HttpClientModule],
})
```

2. Criar serviço para posts (post.service.ts):

```
bash

ng generate service services/post
```

Passo 2: Implementar CRUD (1h30)

1. Serviço completo (post.service.ts):

```
typescript

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { catchError } from 'rxjs/operators';

@Injectable({ providedIn: 'root' })
export class PostService {
  private apiUrl = 'https://jsonplaceholder.typicode.com/posts';

  constructor(private http: HttpClient) {}

  getPosts() {
    return this.http.get<any[]>(this.apiUrl);
  }

  createPost(post: { title: string; body: string }) {
    return this.http.post(this.apiUrl, post);
  }

  updatePost(id: number, post: any) {
    return this.http.put(`${this.apiUrl}/${id}`, post);
  }
}
```

```

deletePost(id: number) {
  return this.http.delete(`${this.apiUrl}/${id}`).pipe(
    catchError(error => {
      console.error('Erro ao deletar:', error);
      throw error;
    })
  );
}
}

```

2. Componente de listagem (post-list.component.ts):

typescript

```

export class PostListComponent implements OnInit {
  posts: any[] = [];

  constructor(private postService: PostService) {}

  ngOnInit() {
    this.postService.getPosts().subscribe(
      posts => this.posts = posts,
      error => console.error('Erro ao carregar posts:', error)
    );
  }

  deletePost(id: number) {
    this.postService.deletePost(id).subscribe(() => {
      this.posts = this.posts.filter(post => post.id !== id);
    });
  }
}

```

Passo 3: Interface de Usuário (1h)

1. Template (post-list.component.html):

html

```

<div *ngFor="let post of posts" class="post-card">
  <h3>{{ post.title }}</h3>
  <p>{{ post.body }}</p>
  <button (click)="deletePost(post.id)">Excluir</button>
</div>

<div *ngIf="posts.length === 0">
  Carregando posts...
</div>

```

2. Estilização (post-list.component.css):

CSS

```
.post-card {
  border: 1px solid #ddd;
  padding: 15px;
  margin: 10px 0;
  border-radius: 8px;
}

button {
  background-color: #ff4444;
  color: white;
  border: none;
  padding: 5px 10px;
  border-radius: 4px;
}
```

3. Componente de criação (post-form.component.ts):

typescript

```
export class PostFormComponent {
  newPost = { title: '', body: '' };

  constructor(private postService: PostService) {}

  onSubmit() {
    this.postService.createPost(this.newPost).subscribe(() => {
      alert('Post criado com sucesso!');
      this.newPost = { title: '', body: '' };
    });
  }
}
```

Passo 4: Interceptor Global (30 min)

1. Criar interceptor (auth.interceptor.ts):

typescript

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const authReq = req.clone({
      setHeaders: { 'Content-Type': 'application/json' }
    });
    return next.handle(authReq);
  }
}
```

2. Registrar no AppModule :

typescript

```
providers: [  
  { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }  
]
```

Tarefa para Casa

1. Implementar atualização de posts:

- Criar componente `post-edit` com formulário pré-preenchido.

2. Adicionar loading state:

- Exibir spinner durante requisições.

Próxima Aula

- **Tópico:** Autenticação JWT com Interceptors.
- **Atividade prática:** Sistema de login com token.

Resultado Esperado

- Aplicação consumindo API REST com:
 - Listagem, criação e exclusão de posts.
 - Tratamento básico de erros.
 - Interceptor para headers.

plaintext

Fluxo HTTP:

Componente → Serviço → Interceptor → API → Resposta

<https://angular.io/generated/images/guide/http/http-request-flow.png> (Fluxo de requisições HTTP no Angular) 🚀